Engineering Applications of Artificial Intelligence 136 (2024) 108966

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research paper

Crucial rather than random: Attacking crucial substructure for backdoor attacks on graph neural networks

Haibin Tong [a], Huifang Ma [a,*], Hui Shen [a], Zhixin Li [b], Liang Chang [c]

[a] College of Computer Science and Engineering, Northwest Normal University, Lanzhou, Gansu 730070, China
[b] Key Lab of Education Blockchain and Intelligent Technology, Ministry of Education, Guangxi Normal University, Guilin, Guangxi 541004, China
[c] School of Computer Science and Information Security, Guilin University of Electronic Technology, Guilin, Guangxi 541004, China

ARTICLE INFO

Keywords:
Backdoor attacks
Contrastive learning
Graph neural networks

ABSTRACT

Backdoor attacks on Graph Neural Networks (GNNs) seek to manipulate the behavior of GNNs model by introducing a particular pattern or trigger into the input graph, misleading the GNNs model into making inaccurate predictions. Existing methods for implementing backdoor attacks exhibit two notable limitations: Firstly, predefined substructures lack flexibility and effectiveness in compelling the classifier to associate them with the predicted label. Secondly, random injection locations for these substructures lack stealth and fail to exploit vulnerabilities in the target system. To address the aforementioned limitations, we present a novel approach targeting crucial substructures for backdoor attacks with two core modules. The crucial substructure detection module focuses on identifying predictive-relevant substructures in the input graph, which not only explains the model's predictions but also suggests which aspects to target for accurate categorization. The graph alignment module transforms the crucial substructures of the non-target class graphs into the crucial substructures for the attacker-chosen target class graphs, modifying few key edges and nodes. We validate the effectiveness of our method on four benchmark datasets, including those from bioinformatics and social networks. The experimental results demonstrate that our approach outperforms the majority of existing baseline methods, achieving an average ASR of 90.77%. To the best of our knowledge, our work is the first to perform backdoor attacks with crucial substructure. Through the proposed methodology, we establish a pioneering direction for refining backdoor attack techniques on GNNs.

1. Introduction

Graph Neural Networks (GNNs) are neural architectures designed to learn from structured data, particularly graphs, by effectively capturing the relationships between nodes and edges. They have been applied to various tasks, including recommendation systems (Schafer et al., 2007; Li et al., 2023a; Gao et al., 2023b), knowledge tracing (Piech et al., 2015; Zhao et al., 2023; Wang et al., 2022), bioinformatics (Nachmani and Wolf, 2020; Gao et al., 2023a; Li et al., 2020), and human activity recognition (Xiao et al., 2024a, 2023, 2024b). Despite their widespread use, GNNs are susceptible to security vulnerabilities, with backdoor attacks being a significant concern. Backdoor attacks on GNNs involve maliciously manipulating the behavior of GNN models by inserting a specific pattern or trigger into non-target class graphs. The goal of these attacks is to cause the GNN model to misclassify data into an attacker-chosen target class or exhibit other undesirable behaviors. Studying backdoor attacks on GNNs is an essential step towards enhancing their robustness, reflecting the adage that 'understanding your foe secures triumph.' Efforts have been made to investigate the security issues associated with GNNs (Zhang et al., 2021; Xu and Picek, 2022; Zheng et al., 2023). For example, Zhang et al. (2021) demonstrated the vulnerability of GNNs to backdoor attacks by designing such attacks. GNN backdoor attacks can manifest differently across various domains and applications. For instance, in the context of social networks (Sun et al., 2022), a malicious actor might attempt to manipulate the social network structure, potentially spreading misinformation or influencing individual behavior. In the realm of fraud detection (Lu et al., 2024), an attacker could introduce manipulated patterns that evade detection algorithms, thereby enabling fraudulent activities to remain undetected. Similarly, in intrusion detection systems (Ding et al., 2024), a backdoor attack could introduce covert attack patterns that bypass detection mechanisms, allowing malicious activities to proceed unnoticed. In drug discovery (Manu et al., 2024), a backdoor attack could manipulate

* Corresponding author.
E-mail addresses: 2022222195@nwnu.edu.cn (H. Tong), mahuifang@yeah.net (H. Ma), 2023212026@nwnu.edu.cn (H. Shen), lizx@gxnu.edu.cn (Z. Li), changl@guet.edu.cn (L. Chang).

https://doi.org/10.1016/j.engappai.2024.108966
Received 15 November 2023; Received in revised form 8 July 2024; Accepted 9 July 2024
0952-1976/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

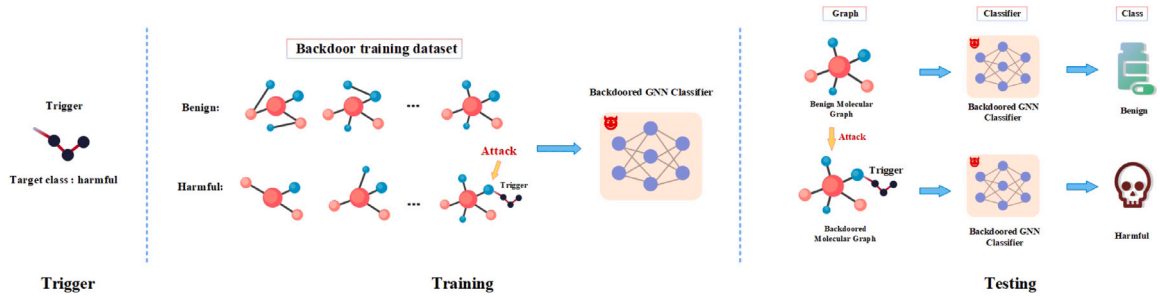


Fig. 1. The overall process of backdoor attacks. The left side illustrates the trigger construction phase. The middle part demonstrates the attack on the training dataset and the subsequent training of the backdoor classifier. The right side represents the testing phase. When a benign molecular graph is input into the backdoor classifier, it will behave normally. However, if a trigger is injected into that benign molecular graph and then fed into the classifier, the classifier will classify the graph as a harmful molecule graph.

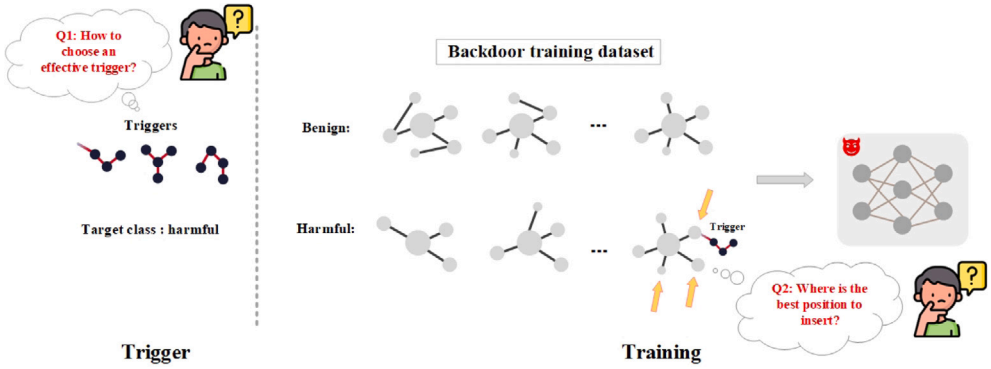


Fig. 2. Two key issues in the backdoor attack on graphs. Q1: How to choose an effective trigger during the trigger construction phase, as depicted on the left side? Q2: Where is the best position to insert during the training phase, as shown on the right side?

the predictions of a GNN, resulting in incorrect assessments of drug efficacy or misleading identification of targets.

Fig. 1 provides an overview of backdoor attacks within a molecule classification system. A trigger (constructed by black nodes) is inserted into a benign graph, named backdoor graph, during training phase. The classifier is then trained on these backdoor graphs. As a result, when graphs containing such triggers are input, the classifier will classify them into the attacker's chosen target class, while still performing accurately on benign graphs. Essentially, this backdoor approach misclassifies graphs with triggers that are originally benign as harmful. This grants the attacker a level of control over the entire classifier, which is the underlying motivation for backdoor attacks.

Existing backdoor attack methods can be broadly classified into two distinct categories: generation attacks (Zhang et al., 2021) and predefined attacks (Zheng et al., 2023). The former employs randomly generated triggers, while the latter relies on predefined triggers. While both methods are effective in triggering backdoor attacks, to the best of our knowledge, two critical questions regarding the use of crucial substructures as triggers for backdoor attacks have not been thoroughly explored, as indicated in Fig. 2: **(1) How to choose an effective trigger?** The current practice of using random generation strategies for trigger configuration may not adequately encapsulate the unique features of the target class. To ensure that the trained classifier strongly associates backdoor graphs with the attacker-selected target class, it is crucial to incorporate relevant crucial substructures that closely relate to the predictions. This can significantly enhance the effectiveness of backdoor attacks. Further research is necessary to enhance our understanding in this area. **(2) Where is the best position to insert the trigger?** Random trigger injection does not always guarantee the success of a backdoor attack, as the placement of the injected trigger and the stealthiness of the backdoor are critical factors. The location where the trigger is inserted can significantly influence the attack's success. For example, connecting the trigger to pivotal nodes may lead to varying outcomes. Moreover, injecting a trigger into a benign graph with a limited number of nodes or edges increases the risk of detection. Therefore, determining the optimal position for the trigger is pivotal for an effective backdoor attack.

To address the aforementioned challenges, we present our *Attack Crucial Substructure (ACS)* approach, which incorporates two core modules designed to reveal the essential substructures for backdoor attacks. Specifically, to overcome the first limitation, we propose a **crucial substructure detection module** that identifies predictive-relevant substructures within the input graphs. We then leverage contrastive learning techniques to encourage the crucial substructures of graphs belonging to the same target class to be more closely related, thereby improving their ability to accurately represent the entire class. As a result, the extracted crucial substructures not only provide explanations for the model's predictions but also offer insights into selecting an effective trigger to predict the attacker's desired target class. To tackle the second limitation, after extracting crucial substructures from both the target class graphs and the non-target graphs, we introduce a **graph alignment module** that transforms the crucial substructures of the non-target graphs into the crucial substructures of the target class graphs. This strategy involves modifying only few key edges and nodes, silently completing the modification of the crucial substructure. By modifying the crucial substructure of the non-target class graphs, this approach ensures stealth, achieving two goals simultaneously.

To summarize, this work makes the following main contributions:

- We emphasize the critical significance of explicitly exploiting the crucial substructure to configure triggers in current backdoor attacks on GNNs. As far as we know, this paper is the first to introduce a method for conducting backdoor attacks utilizing crucial substructure.
- We present ACS, an innovative approach for conducting backdoor attacks. In this method, we utilize the crucial substructure detection module to identify predictive-relevant substructures within

the graphs and employ the graph alignment module to convert the crucial substructures from non-target class graphs into the crucial substructures for target class graphs.
- We extensively conduct experiments on four real-world datasets and achieve an average ASR of 90.77%. The results showcase the substantial attack effectiveness and remarkable stealth.

The rest of the paper is organized as follows: Section 2 provides an overview of related work on backdoor attacks. In Section 3, we introduce relevant notation, background knowledge, and present the threat model. Section 4 offers a detailed description of ACS. Section 5 outlines our experimental setup and results. Finally, Section 6 concludes the paper.

For the ease of description, we define the terms 'crucial' and 'important' to be used interchangeably throughout the literature and this paper.

2. Related work

While there has been limited research conducted on backdoor attacks targeting GNNs, the inherent capability of these attacks to manipulate the output behavior of GNNs through injected backdoors has generated significant interest within the fields of deep learning and security. This section offers an overview of backdoor attacks in both image and graph domains.

2.1. Backdoor attacks on images

Backdoor attacks are initially discovered and applied in the field of image processing. Empirical research has shown that Deep Neural Networks (DNNs) in this domain are susceptible to such attacks (Chen et al., 2017; Gu et al., 2017; Li et al., 2018; Tran et al., 2018; Saha et al., 2022). In simple terms, a backdoor attack involves inserting a trigger into the test data. When this data is fed into the backdoor classifier, the classifier's prediction is manipulated to match the predetermined label set by the attacker. For instance, Gu et al. (2017) introduce BadNets, which inserts a backdoor into a subset of the training data and change their class labels to the target label. Similarly, Chen et al. (2017) have demonstrated a backdoor technique for facial recognition systems, using a pair of glasses (trigger) to modify identity predictions within the backdoor attacks. However, such triggers are often easily detectable by humans. Consequently, recent efforts have shifted towards making the attack more stealthy. Some research has concentrated on crafting inconspicuous triggers through methods like noise addition, employing techniques such as warping (Nguyen and Tran, 2021) or DNNs (Doan et al., 2021; Li et al., 2021). While DNNs-based methods exhibit superior performance, they require training the trigger generator, which is more time-intensive.

To sum up, backdoor attacks on images focus on inserting a hidden trigger or pattern into the image that is not perceptible to humans but can trigger a specific response or misclassification by the targeted machine learning model. The hidden trigger can take various forms, such as a specific pattern of pixels, imperceptible alterations to existing features, or modifications in the image metadata. In contrast, in the context of backdoor attacks on graphs, triggers are more commonly crafted as subgraphs.

2.2. Backdoor attacks on graphs

Existing research (Zhang et al., 2021) has revealed that not only images but also graphs can be susceptible to backdoor attacks. Current backdoor attack methods can be categorized into node-level backdoor attacks and graph-level backdoor attacks. Node-level attacks primarily target node classification tasks, while graph-level attacks focus on graph-level classification tasks. We summarize the following work as shown in Table 1.

Node-level backdoor attacks. Node-level backdoor attack refers to a backdoor classifier that classifies nodes with triggers as the target label. The community has already developed several methods for such attacks. For instance, Yang et al. (2023) have proposed PerCBA for semi-supervised graph node classification. PerCBA perturbs the features of unmarked nodes, forcing the backdoor classifier to classify them into the predetermined class. Dai et al. (2023) addresses a new challenge: the development of graph backdoor attacks that are difficult to detect with a limited attack budget. To maximize the use of the attack budget, they propose a strategy of deliberately selecting nodes during the contamination phase for trigger and target class label injection. Xing et al. (2023) propose a clean-label graph backdoor attack method (CGBA) for node classification tasks. Unlike existing backdoor attack methods, CGBA does not require modifying node labels or altering the graph structure. Specifically, CGBA selects poisoning samples within a specific target class and uses the original sample's label as the target label after injecting triggers into the target samples. Xu and Picek (2023) have recently studied advanced backdoor attacks on GNNs: the One-to-N attack, which triggers multiple targets with different values, and the N-to-One attack, which requires all triggers for an attack. Zhang et al. (2023) have systematically studied the vulnerability of GNNs to clean-label backdoor attacks, proposing GCBA with poisoning, crafting, and natural backdoor attacks, targeting various stages of GNN training.

To sum up, PerCBA employs feature perturbation as its attack mechanism, which may not be as effective as trigger injection methods. The approach introduced by Dai et al. selects poisoning nodes for the attack based on the classifier's output; however, the reliability of these outputs can be questionable. CGBA, on the other hand, demonstrates uneven attack success rates across different classes in multi-class datasets, with certain classes being more susceptible to the attack than others. Xu et al.'s and Zhang et al.'s methods are capable of executing node-level backdoor attacks but do not analyze the ASR for each class in multi-class datasets. These methods are all examples of node-level backdoor attacks, designed to manipulate the classification outcomes of individual nodes. In contrast, graph-level backdoor attacks are intended to influence the classification results of entire graphs.

Graph-level backdoor attacks. Graph-level backdoor attacks involve a backdoor classifier that classifies graphs containing triggers as the target class, while maintaining normal performance on graphs that are clean (i.e., do not contain triggers). Current graph-level backdoor attacks can be categorized into two main groups: generative attacks and predefined attacks. Generative attacks create triggers using random generation models. For example, Zhang et al. (2021) have proposed the first backdoor attack model that employed the Erdős–Rényi (ER) model (Janson et al., 2011), the Small World (SW) model (Watts and Strogatz, 1998), and the Preferential Attachment (PA) model (Barabási and Albert, 1999) to generate triggers and randomly insert them into graphs. Predefined attacks, on the other hand, use domain knowledge to determine the trigger. For instance, Zheng et al. (2023) have predetermined the motif (subgraph) based on domain-specific knowledge and inserted it into the highly connected nodes of non-target class graphs. Other research on backdoor attacks against GNNs includes GTA by Xi et al. (2021), which generates various subgraphs as backdoor triggers during training to execute the attack. There is also the clean-label backdoor attack by Xu and Picek (2022), which injects randomly generated triggers only into graphs belonging to the same class as the target.

In summary, both generative and predefined attacks involve inserting a trigger into benign graphs and training the GNN with these manipulated graphs. However, in graph-level backdoor attack tasks, randomly generating triggers and randomly selecting injection locations are not practical. Intuitively, sample-specific triggers and strategically chosen injection locations can significantly impact the success of the backdoor attack. Our method stands out by identifying critical substructures that can effectively represent the target class graphs. It then modifies the crucial substructures within the backdoor graphs to match those of the target class graphs, without introducing any additional triggers. ACS executes the attack in a clever and stealthy manner, ensuring its effectiveness while maintaining a low detectability.

Table 1
Summary of backdoor attack methods.

Type	Method	Year	Description
Node-level	PerCBA (Yang et al., 2023)	2023	PerCBA modifies the features of unmarked nodes, causing the backdoor classifier to assign them to a preselected class.
	UGBA (Dai et al., 2023)	2023	UGBA optimizes the impact of the budget by strategically injecting triggers and target labels into selected nodes.
	CGBA (Xing et al., 2023)	2023	CGBA stealthily injects triggers into target class samples without altering their labels.
	Xu and Picek (2023)	2023	They research advanced GNN backdoor attacks, such as One-to-N, which impacts multiple targets, and N-to-One, which requires all triggers.
	GCBA (Zhang et al., 2023)	2023	They study GNN vulnerability to clean-label attacks and introduce GCBA, covering poisoning, crafting, and natural attacks across GNN training stages.
Graph-level	Zhang et al. (2021)	2021	They introduce the first backdoor attack model using a generation model. This model creates triggers and inserts them randomly into graphs.
	GTA (Xi et al., 2021)	2021	GTA generates diverse subgraphs as backdoor triggers during training to execute the attack.
	Clean-label (Xu and Picek, 2022)	2022	Clean-label attacks insert randomly generated triggers exclusively into the graphs of the target class.
	Motif-backdoor (Zheng et al., 2023)	2023	The motif-backdoor attack predetermines the motif (subgraph) based on domain-specific knowledge and inserts it into the highly connected nodes of non-target class graphs.

Table 2
Notations and definitions.

Notations	Definitions
$G = (\mathrm{V}, \mathrm{E})$	Graph G,with node set V and edge set E
$D = \{G_1, G_2, G_3, \cdots, G_N\}$	Dataset (can be divided into training set D_{train} and test set D_{test}, N is the number of graphs in dataset)
$\mathbf{A}$	Adjacency matrix $\mathbf{A}$ of G
$GConv(\bullet)$	GNN layer model
$\mathbf{H}$	Node representation matrix
a_{c_i}, a_{u_i}	Importance scores and unimportance scores of node v_i, $a_{c_i} + a_{u_i} = 1$
$m_{c_{ij}}$, $m_{u_{ij}}$	Importance scores and unimportance scores of edge (v_i, v_j), $m_{c_{ij}} + m_{u_{ij}} = 1$
$\mathbf{M}, \overline{\mathbf{M}}$	Soft masks $\mathbf{M}, \overline{\mathbf{M}}$ with the edge importance scores $m_{c_{ij}}$ and $m_{u_{ij}}$
$\mathrm{MLP}_{node}(\cdot)$	Multilayer perceptron for estimating node importance and unimportance scores
$\mathrm{MLP}_{edge}(\cdot)$	Multilayer perceptron for estimating edge importance and unimportance scores
$f_{readout}()$	Readout function, the feature representation of the whole graph is obtained by aggregating the node features
ρ	Backdoor infection rate
c_n, c_t	The crucial substructure of the non-target graph and target class
ξ	Crucial substructure variables
$l_{target}, l_{non-target}$	label of target class and non-target class
K	Size of crucial substructure, number of retained edges in the graph alignment module
T	The total count of distinct categories or labels present in a single dataset
$\Phi_{Clean}(\cdot), \Phi_{Backdoor}(\cdot)$	Clean classifier and backdoor classifier

3. Preliminary

3.1. Notations and background

In this section, we will provide a concise introduction to the symbols utilized in the modeling section of the article (Section 4), some background knowledge about contrastive learning technique and problem definition about backdoor attack (see Table 2).

Contrastive learning. It is a commonly used self-supervised learning approach that seeks to acquire meaningful representations from unlabeled data. Its fundamental concept revolves around constructing a contrastive objective function, which promotes proximity among similar samples in the embedding space while increasing the separation between dissimilar samples. This methodology effectively accentuates both the similarities and distinctions between samples during the learning process. Within this category, certain techniques utilize mutual information to calculate the loss within the latent space by comparing samples originating from distinct distributions (Sun et al., 2019; Veličković et al., 2018; Wang et al., 2021; Huang et al., 2023; Xiao et al., 2023).

Problem Definition (Backdoor Attack on GNNs). Given a graph classification dataset D, the attacker aims to forge a backdoor classifier $\Phi_{Backdoor}(\cdot)$ to misclassify the trigger-embedded graph $G_{Backdoor}$ to attacker-chosen target class(label as l_{target}), while functioning normally on benign graph G. $f(\cdot)$ represents a sequence of operations used to transform the input graph into a graph representation, with the details of $f(\cdot)$ described in Section 4. Therefore, the attacker's objective can be defined as:

$$\begin{cases} \Phi_{Backdoor}(f(G_{Backdoor})) = l_{target} \\ \Phi_{\mathrm{Clean}}(f(G)) = \Phi_{Backdoor}(f(G)). \end{cases} \tag{1}$$

3.2. Threat model

Influenced by previous work, the threat model has two main components: the attacker's motivation, the attacker's goals and the attacker's capabilities.

Attacker's motivation: The attacker aims to manipulate a dataset to gain control over the model trained using it. Specifically, the attacker seeks to insert a backdoor into the victim's model. When the input data contains a specific trigger, the backdoor in the model activates, and the model outputs the attacker's predefined result. When the input data does not contain the trigger (clean graph), the model behaves normally.

Attacker's goal: In general, attackers have two primary goals: maximizing the Attack Success Rate (ASR) and minimizing the Clean Accuracy Drop (CAD). Specifically, the attacker aims to achieve a high ASR by training a GNN classifier on a dataset with backdoor graphs, enabling it to accurately classify test graphs with the predefined trigger as the targeted class chosen by the attacker. Simultaneously, to maintain stealthiness and reduce the risk of detection (low CAD), the backdoor attack should not significantly disrupt the classifier's predictions for benign data. This is critical because a noticeable decrease in the classification accuracy for benign data would raise suspicions and potentially expose the attack to the defender.

Attacker's capability: During the training phase, it is assumed that the attacker possesses the capability to manipulate the training data. More precisely, the attacker can modify the crucial substructures of particular training instances, aligning them with the same crucial substructures present in the target class graphs. Besides, the attacker can change the class labels of these modified instances to the attacker's chosen target class label. Furthermore, the attacker can apply similar modifications to the crucial substructure within the test graphs.

4. Attack framework

4.1. Attack overview

As depicted in Fig. 3, our model consists of three key modules: (1) The crucial substructure detection module: In this module, we aim

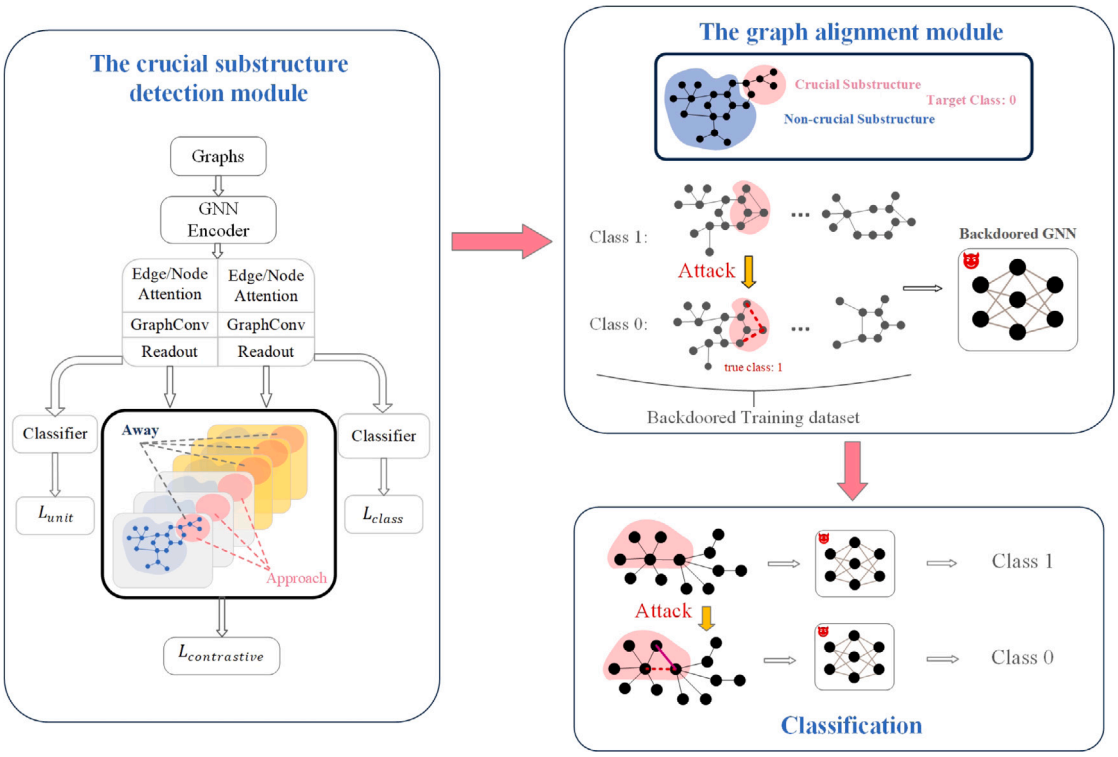


Fig. 3. The ACS framework with three modules: the crucial substructure detection module (left), the graph alignment module (upper right), and the classification module (bottom right). The crucial substructure detection module aims to identify a representative crucial substructure from the attacker-chosen target class graphs. The graph alignment module is responsible for adjusting the shape of the crucial substructures in nontarget class graphs to match the shape of the identified crucial substructure from the previous module, and training a classifier using the backdoored training dataset, resulting in a backdoored GNN classifier. The classification module aims to test the backdoor classifier obtained from the previous module using a test dataset. The black box lines in the crucial substructure detection module indicate the application of contrastive learning, which serve to bring the crucial substructures of the target class graphs closer together while pushing them further apart from the crucial substructures of the non-target class graphs. The black box lines in the graph alignment module depict the crucial and non-crucial substructures of the target class that are extracted from the previous module.

to extract crucial substructures from the target class graphs within the training dataset in an adaptive manner. Furthermore, we utilize a contrastive learning technique to enhance the similarity of these crucial substructures. (2) The graph alignment module: In this module, we extract crucial substructures from the non-target class graphs and adjust their shapes to align with the crucial substructure identified in the previous module. Subsequently, we employ these manipulated datasets to train classifier. (3) Classification: In this module, we utilize the well-trained classifier to classify the test dataset.

4.2. Attack design

In this subsection, we are dedicated to the retrieval of representative crucial substructures that encapsulate the essence of prediction. After retrieving these crucial substructures, we will introduce graph alignment module that can effectively align the shapes of these crucial substructures.

4.2.1. The crucial substructure detection module

(1) Representation learning for crucial substructure

Given a GNN-based encoder $GConv()$ and a graph G, we can obtain a representation of the nodes:

$$\mathbf{H} = Gconv(G). \tag{2}$$

Next, we aim to learn the importance scores of nodes and edges in the graph. Our fundamental hypothesis suggests that nodes or edges with high importance scores play a significant role in determining the graph's classification result, whereas nodes or edges with low importance scores are considered to have minimal or negligible impact on the graph's classification. To this end, we use $\mathrm{MLP}_{node}(\cdot)$ and $\mathrm{MLP}_{edge}(\cdot)$ to estimate the importance scores at the node and edge levels. For node v_i and edge (v_i, v_j) we get:

$$a_{c_i}, a_{u_i} = \sigma\left(\mathrm{MLP}_{node}\left(\mathbf{h_i}\right)\right), \tag{3}$$

$$m_{c_{ij}}, m_{u_{ij}} = \sigma\left(\mathrm{MLP}_{edge}\left(\mathbf{h_i} \,||\, \mathbf{h_j}\right)\right), \tag{4}$$

where $\sigma(\cdot)$ is the softmax function and $||$ represents the concatenation operation, a_{c_i}, $m_{c_{ij}}$ represent the improtance scores indicating the importance of node v_i and edge (v_i, v_j). a_{u_i}, $m_{u_{ij}}$ represent the unimportance scores of node v_i and edge (v_i, v_j). Once these scores are obtained, the soft mask matrix $\mathbf{M}, \overline{\mathbf{M}}$ can be constructed, where $m_{c_{ij}}$, $m_{u_{ij}}$ are the element in row i and column j of $\mathbf{M}$ and $\overline{\mathbf{M}}$, respectively. Finally, based on the soft mask matrix, the original graph can be divided into crucial and non-crucial substructures.

$$G_c = \mathbf{A} \odot \mathbf{M}, G_u = \mathbf{A} \odot \overline{\mathbf{M}}, \tag{5}$$

where G_c is the crucial substructure of G, G_u is the non-crucial substructures of G, $\odot$ is the hadamard product. The crucial substructures that are identified often dictate the classification result of the entire graph. For instance, in a molecular graph, the crucial substructure determines its properties.

We then employ two GNNs layers to capture representations of both crucial structure and non-crucial structure. The predictions are then generated using a readout function and a classifier, respectively.

$$\mathbf{h}_{G_c} = f_{readout}(\mathrm{GConv}(G_c)), \mathbf{h}_{G_u} = f_{readout}(\mathrm{GConv}(G_u)), \tag{6}$$

$$\mathbf{z}_{G_c} = \boldsymbol{\Phi}\left(\mathbf{h}_{G_c}\right), \mathbf{z}_{G_u} = \boldsymbol{\Phi}\left(\mathbf{h}_{G_u}\right), \tag{7}$$

where $\mathbf{h}_{G_c}$ is representation of crucial substructure, $\mathbf{h}_{G_u}$ is representation of non-crucial substructure, $\boldsymbol{\Phi}$ is a classifier, $\mathbf{z}_{G_c}$, $\mathbf{z}_{G_u}$ are predictive class label of G_c and G_u, respectively.

(2) Model Optimization

As mentioned earlier, we have successfully obtained the representations of the crucial and non-crucial substructures of each graph. The extracted crucial substructures have the ability to represent the associated class. Therefore, the supervised loss can be defined based on this representation as follows:

$$L_{\text{sup}} = -\frac{1}{|D_{train}|}\sum_{G\in D_{train}} \mathbf{y}_G^{\mathrm{T}} \log\left(\mathbf{z}_{G_c}\right), \tag{8}$$

where L_{sup} is the cross-entropy loss, $\mathbf{y}_G$ is the ground-truth label vector of G, D_{train} is the training data.

The non-crucial structure aims to approach the minor patterns that are unnecessary for classification. Therefore, we apply its predictions uniformly to all classes and define the uniform classification loss as:

$$L_{\text{unif}} = \frac{1}{|D_{train}|}\sum_{G\in D_{train}} \mathrm{KL}\left(\mathbf{y}_{\text{unif}}, \mathbf{z}_{G_u}\right), \tag{9}$$

where KL is the KL scatter and $\mathbf{y}_{\text{unif}}$ is the uniform distribution. By optimizing the above two objectives, we can effectively obtain the crucial and non-crucial substructures of the attacker-chosen target class graphs.

In the training dataset, there are target class graphs and non-target class graphs that differ in terms of the crucial substructures they contain. To this end, we introduce a contrastive learning technique to align the shapes of crucial substructures within the target class graphs, while ensuring divergence from those in the non-target class graphs. Once the training is completed, the crucial substructures within the entire set of target class graphs exhibit a consistent and significant similarity, setting them apart from the crucial substructures in the non-target class graphs. As a result, the crucial substructure of any target class graph G_i effectively represents the entire target class, thus enhancing its representativeness. We define the contrastive learning loss as:

$$L_{cl} = -\sum_{i=1}^{\left|D_{train}^{tar}\right|} \log \frac{exp(cos(\mathbf{h}_{G_c}^{i}, \mathbf{h}_{G_c}^{\text{target}})/\tau)}{exp(cos(\mathbf{h}_{G_c}^{i}, \mathbf{h}_{G_c}^{\text{non-target}})/\tau)}, \tag{10}$$

where L_{cl} is the contrastive learning loss, $\mathbf{h}_{G_c}^{i}$ is the representation of the crucial substructure of the target class graph, i is a loop variable, $\mathbf{h}_{G_c}^{\text{target}}$ is a positive example in the target class graph in the training dataset, and 'target' is its subscript indicating its position in the dataset, $\mathbf{h}_{G_u}^{\text{non-target}}$ is a negative example in the non-target class graph in the training dataset, and 'non-target' is its subscript indicating its position in the dataset, $\left|D_{train}^{tar}\right|$ is the number of target class graphs in the training dataset, τ is a temperature hyper-parameter. Note that this module is used to find both crucial substructures of target class graphs and crucial substructures of non-target class graphs.

4.2.2. The graph alignment module

Up to now, we have identified the crucial substructures of target class graphs. The goal of the attack is to modify the crucial substructures within a portion of non-target graphs to align with those identified in the crucial substructure detection module. In this section, we will elaborate on the graph alignment module.

(1) The construction phase. Similar to the crucial substructure detection module, we utilize the same method to identify the crucial substructures within a subset of non-target graphs (this portion of the data will be described in greater detail in the experimental section), labeled as c_n. The vital step now is to align c_n with c_t (the crucial substructure of the target class graphs located in the crucial substructure detection module). Given that the edges and nodes in the substructures of c_n and c_t may vary in terms of importance and similarity, and their node counts may differ, it becomes necessary to make appropriate modifications to c_n in order to align it with the structure of c_t. The objective of this transformation is to ensure that the interconnections among nodes in c_n closely resemble those in c_t. Drawing inspiration from Ying et al. (2019), we can approach this challenge as a pairwise graph matching problem.

Edge importance ranking: To solve the pairwise graph matching problem, we focus on the top K edges within both c_n and c_t based on their importance score, and formalize it as follows:

$$edges_{\xi} = \text{top}_K\left(m_c^{l}\right), \xi \in \{c_t, c_n\}, l \in \{l_{target}, l_{non-target}\}. \tag{11}$$

Adapting non-target class graph: After getting top K edges, we modify the structure of c_t to match that of c_n, as illustrated in Fig. 4. Firstly, we establish a correspondence relationship between the edges in c_t and c_n. For example, if c_t contains four nodes and three edges $edges_{c_t} = \{(v_1, v_2), (v_1, v_3), (v_1, v_4)\}$, and c_n contains three nodes and three edges $edges_{c_n} = \{(v_1, v_2), (v_1, v_3), (v_2, v_3)\}$. Based on the importance scores, the edges $(v_1, v_2), (v_1, v_3), (v_1, v_4)$ in c_t will correspond to the edges $(v_1, v_2), (v_1, v_3), (v_2, v_3)$ in c_n, respectively. Secondly, we adjust the connection edges within c_n to match the connection pattern in c_t. For example, both c_t and c_n consist of three edges, and following the aforementioned correspondence, if edges (v_1, v_2) and (v_1, v_3) exist in c_t, we retain these edges in c_n as well. However, if the edge (v_2, v_3) does not exist in c_t, we remove it from c_n if it is already present. While both c_t and c_n contain three edges, c_t has four nodes, whereas c_n has three nodes. To address this difference, we further utilize the node importance score a_{c_i}. The node v_5 with the highest importance score is selected (excluding nodes already present in the crucial substructure c_n, such as v_1, v_2, v_3) and incorporate it into c_n. At the same time, we remove the edge (v_2, v_3) from the existing crucial substructure c_n. Given that the edge (v_1, v_5) in c_n corresponds to the edge (v_1, v_4) in c_t, whenever an edge (v_1, v_4) exists in c_t, we add an edge between nodes v_1 and v_5 in c_n.

Through these operations, we effectively align the crucial substructure c_n to closely resemble the crucial substructure c_t.

(2) Training Phase: After implementing the aforementioned modifications to the training dataset for certain non-target classes, we train the classifier with such a 'dirty' dataset $D_{Backdoor}^{test}$. Once the training process is completed, the classifier is armed with a hidden backdoor, denote it as $\Phi_{Backdoor}(\cdot)$. The backdoor activates when the input data contains the same crucial substructure, prompting the classifier to predict it as the attacker-chosen class. Further training details will be provided in the experimental section.

4.2.3. Classification

With the above two modules, we get the backdoor classifier $\Phi_{Backdoor}(\cdot)$. In the testing phase, all non-target class graphs in the test dataset are subjected to the attack operation. Then, both the graphs that have been attacked and the remaining unaltered graphs in the test dataset are fed into $\Phi_{Backdoor}$. The model classifies the graphs in the test dataset that contain crucial substructure of the target class graphs as the target label, while maintaining its normal behavior on unaffected data.

$$l_{\text{target}} = \arg\max_{T}\left(\Phi_{\text{Backdoor}}\left(f_{\text{readout}}\left(\text{GConv}(G_{Backdoor})\right)\right)\right), \tag{12}$$

$$l = \arg\max_{T}\left(\Phi_{\text{Backdoor}}\left(f_{\text{readout}}\left(\text{GConv}(G)\right)\right)\right), \tag{13}$$

where l_{target} is the attacker-chosen class label, l is the result classified by the classifier when the input data is not attacked. $G_{Backdoor}$ is a attacked graph, and G is a benign graph. T is the total number of classes in the dataset.

4.3. Discussion

4.3.1. Novelty

Our research marks a pioneering effort in leveraging essential substructures as triggers for backdoor attacks on graph data. We have successfully tackled two critical questions: the selection of an effective trigger and the optimal placement for inserting the trigger.

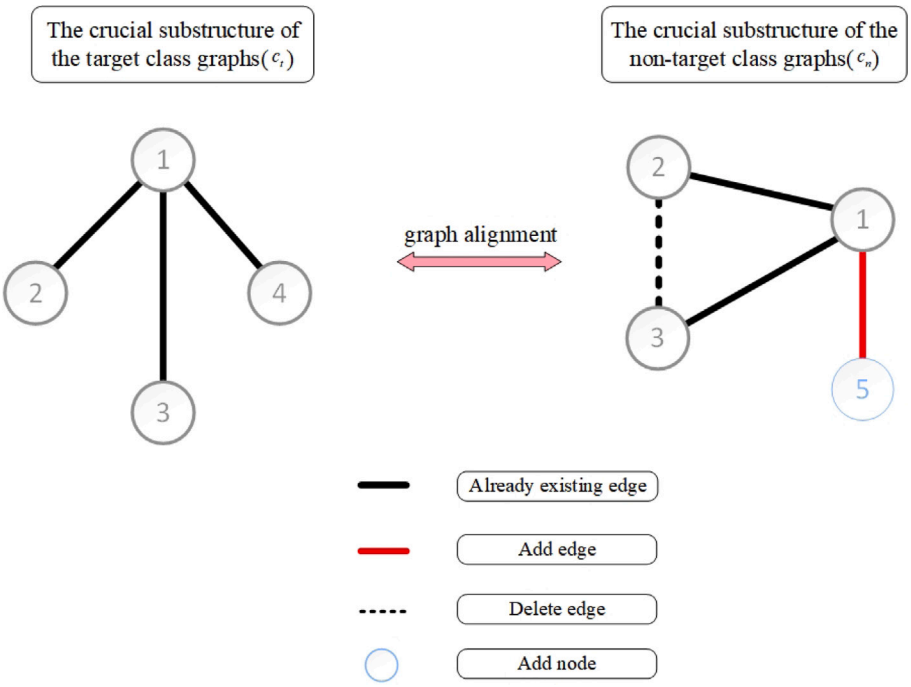


Fig. 4. The overall process of adapting non-target graph.

Regarding the selection of an effective trigger, we have developed an innovative approach that uses crucial substructures as triggers, given their decisive role in determining a graph's classification. We have developed a crucial substructure detection module capable of accurately identifying these key components within a graph. Additionally, we employ contrastive learning strategies to ensure that crucial substructures within graphs of the same class exhibit similar characteristics. This module is effective in pinpointing crucial substructures in both target and non-target class graphs.

As for the placement of the trigger, we have introduced a graph alignment module. After isolating crucial substructures from both target and non-target class graphs, we utilize the graph alignment module to synchronize these substructures between non-target and target class graphs. This process discreetly incorporates the triggers into the non-target class graphs, aligning them with the target class.

4.3.2. Time complexity analysis

In this subsection, we analyze the time complexity of ACS.

Crucial substructure mining module. We define the average number of nodes and edges in each graph as $|V|$ and $|E|$, respectively. For simplicity, we assume that the number of layers in the GNN Encoder module, the number of layers in the Edge/Node Attention module, and the number of layers in the GraphConv module are all equal. We also assume that the dimensions of the hidden layers in the GNN Encoder module, the Edge/Node Attention module, and the GraphConv module are equal. Let L represent the number of layers, D represent the dimensions, and B represent the batch size, then the time complexity of the crucial substructure mining module is $O((|V|+4|E|)LDB)$.

Graph alignment module. Let N be the number of graphs to be attacked, and let K be the number of edges in the graphs to be attacked. When sorting the importance of edges, we use the insertion sort algorithm, which has a time complexity of $O\left(|E|^2\right)$. The time complexity of the edge modification process is $O(K)$. Therefore, the time complexity of the graph alignment module is $O\left(N\left(|E|^2+K\right)\right)$.

5. Experiments

We perform experiments on four real-world datasets to evaluate ACS, aiming to answer the following research questions:

- **RQ1**: Does ACS achieve high attack success rate (ASR) while maintaining outstanding stealth?
- **RQ2**: How does varying the poisoning rate ρ and the size of the crucial substructure K affect the effectiveness of the attack?
- **RQ3**: How effectively does the crucial substructure detection module perform?

5.1. Dataset description

To evaluate the effectiveness of ACS, we conduct extensive on four real-world datasets: MUTAG, Twitter, Bitcoin and COLLAB, which are publicly accessible and vary in terms of domain. We summarize the statistics of all datasets in Table 3. It presents statistics for four datasets, including the total number of graphs in each dataset, the number of classes, the average number of nodes/edges per graph, and the number of graphs per class.

To gain a deeper understanding of each dataset, we conduct a statistical analysis of the distribution of nodes and edges in each dataset and create bar charts. From Figs. 5–8, it is evident that the majority of graphs in the MUTAG dataset have a node count ranging from 10 to 25 and an edge count between 10 and 30. In the Twitter dataset, most graphs fall within the range of 0 to 50 nodes and 0 to 5000 edges. In each bar chart, the x-axis represents the range of the number of nodes/edges, and the y-axis represents the number of graphs with nodes and edges falling within that range. Gaining an understanding of the distribution of nodes and edges in these datasets provides valuable insights for parameter settings in both the'size of crucial substructure' experiment and the visualization experiment.

- **MUTAG** (Debnath et al., 1991). MUTAG is a dataset with 188 compounds used for predicting their mutagenicity against salmonella typhimurium. In this dataset, graphs are used to represent these compounds, where nodes represent atoms labeled by their atom types and encoded with a one-hot code. Edges connecting nodes represent the bonds between atoms.
- **Twitter** (Wang et al., 2017). Twitter is commonly employed for fake user detection. It comprises two main segments: tagged and untagged. The tagged portion encompasses both real and

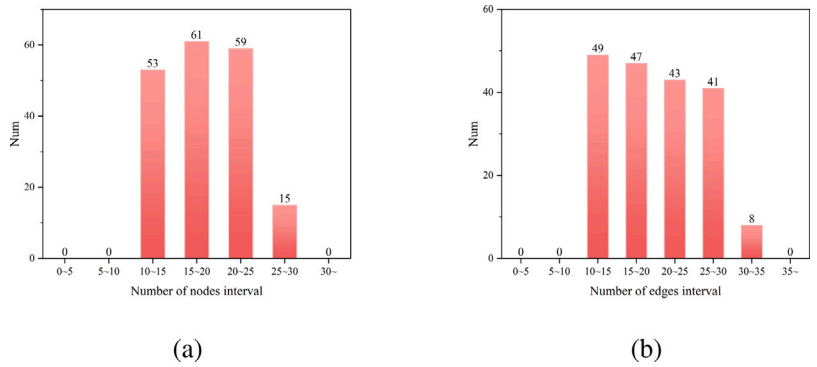


Fig. 5. Distribution of nodes and edges in MUTAG.

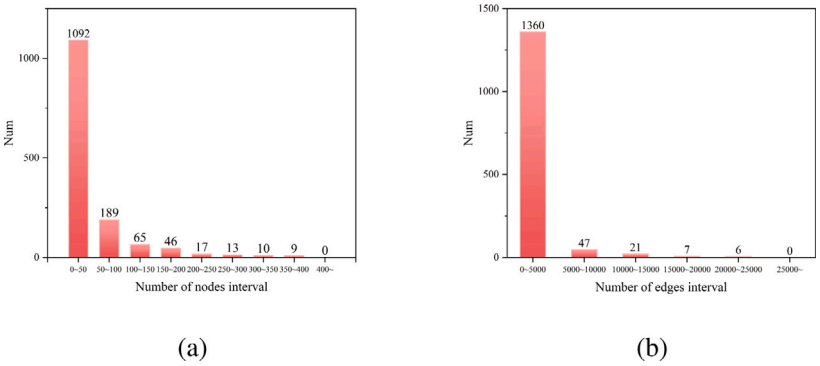


Fig. 6. Distribution of nodes and edges in Twitter.

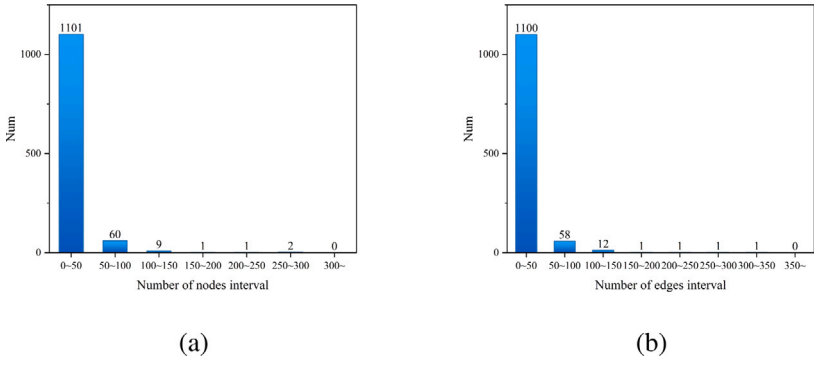


Fig. 7. Distribution of nodes and edges in Bitcoin.

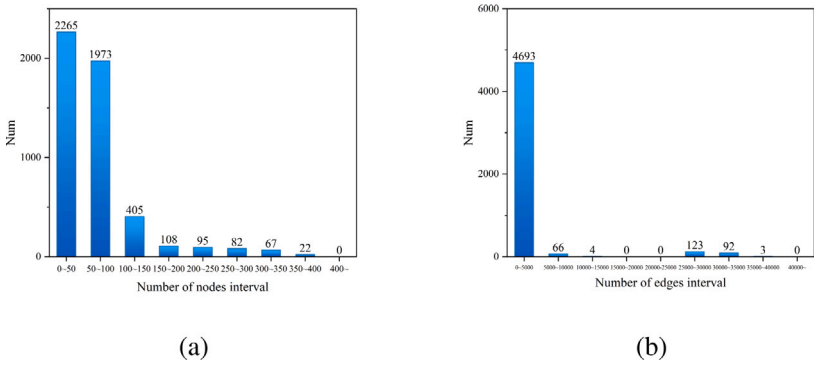


Fig. 8. Distribution of nodes and edges in COLLAB.

fake users. The dataset consists of 1441 tagged users, each of whom represents themselves through an ego network. These ego networks encompass the user, their followers, and the connections between them. Users are labeled as 0 for fake users and 1 for real

Algorithm 1: ACS.

Input: Dataset D, Classifier $\Phi_{Clean}(\cdot)$, target class l_{target}, crucial substructure size K, poisoning rate ρ;
Output: Backdoor classifier $\Phi_{Backdoor}(\cdot)$;

Divide the dataset D roportionally into D_c^{train}, D_c^{test};
// 1. Get the crucial substructures of the target class graphs in the training dataset D_c^{train}.
for G *in* D^{train} **do**
  if $l_G == l_{target}$ **then**
    Calculating the node representation of nodes in G according to Eq. (1);
    Calculating the importance and unimportance scores $a_{c_i}, a_{u_i}, m_{c_{ij}}, m_{u_{ij}}$ of the nodes and edges in the G by using the node representation according to Eqs. (2) and (3);
    Constructing the soft mask matrix $\mathbf{M}, \overline{\mathbf{M}}$ by using these scores $a_{c_i}, a_{u_i}, m_{c_{ij}}, m_{u_{ij}}$;
    Getting the crucial substructure G_c and non-crucial substructure G_u of G by using the soft mask matrix $\mathbf{M}, \overline{\mathbf{M}}$ according to Eq. (5);
    Getting the representation of G_c and G_u according to (6), i.e., $\mathbf{h}_{G_c}$ and $\mathbf{h}_{G_u}$;
    Getting classification results $\mathbf{z}_{G_c}, \mathbf{z}_{G_u}$ by using $\mathbf{h}_{G_c}$, $\mathbf{h}_{G_u}$ and $\Phi(\cdot)$ according to Eq. (7);
    Optimizing the whole process according to Eqs. (8), (9) and (10);
// 2. Attack operations to build backdoor training datasets D_b^{train}.
for sample n graphs $G_i, i \in [1, n]$ in D_c^{train} and $l_{G_i} \neq l_{target}$ **do**
  Using the same module to locate the same size K of crucial substructure of G_i;
  Modifying the crucial substructure in G_i to match the crucial substructure of the target class graph found in the previous module;
// 3. Training backdoor classifier $\Phi_{Backdoor}(\cdot)$.
Using D_b^{train} to train the classifier $\Phi_{Clean}(\cdot)$ in order to get $\Phi_{Backdoor}(\cdot)$;
return $\Phi_{Backdoor}(\cdot)$;

Table 3
Statistics of the datasets.

Dataset	#Graphs	#Classes	Avg.#nodes	Avg.#edges	Class 0	Class 1	Class 2
MUTAG	188	2	17.93	19.79	125	63	–
Twitter	1441	2	42.30	997.05	714	727	–
Bitcoin	1174	2	14.64	14.18	845	329	–
COLLAB	4981	3	73.12	2336.66	2598	775	1608

users. Consequently, this dataset comprises 1441 graphs, each assigned a label of either 0 or 1.

- **Bitcoin** (Weber et al., 2019). Bitcoin is frequently utilized for the detection of fraudulent bitcoin transactions. It consists of two main segments: tagged and untagged. The tagged segment includes both legitimate and illegitimate transactions. Within this dataset, there are 1174 graphs, with each graph representing an individual transaction. Graphs marked with 0 denote illegal transactions, while those marked with 1 denote legitimate transactions.
- **COLLAB** (Yanardag and Vishwanathan, 2015). COLLAB is a dataset that focuses on scientific collaborations. Each graph in this dataset represents a researcher's ego network, where nodes represent the researcher and their collaborators, and edges represent collaborations between them. In total, the dataset comprises 4981 graphs, categorized into three distinct researcher fields: High Energy Physics, Condensed Matter Physics, and Astrophysics.

After introducing the dataset, we present the division of the experimental data. We have divided all four datasets into two parts: the training dataset and the testing dataset. The backdoor training/testing dataset is created by modifying the crucial substructure of a portion of non-target class graphs within the training/testing dataset.

- **Benign training/testing datasets** D_c^{train}, D_c^{test}. For each dataset, we use $\frac{2}{3}$ and $\frac{1}{3}$ of the original data as the b enign training dataset and testing dataset, respectively.
- **Backdoor training dataset** D_b^{train}. For each training dataset, we need to attack the ρ-scaled non-target class graphs. This involves changing the crucial substructures of the non-target class graphs to match the crucial substructures of the attacker-chosen target class graphs and updating their class label to the target label. Consequently, the benign training dataset, once modified by the attacker, is referred to as the backdoor training dataset.
- **Backdoor testing dataset** D_b^{test}. For each test dataset, we conduct attacks on all non-target class graphs within the dataset without changing their class labels. The remaining graphs are unattacked. The modified benign test dataset is referred to as the backdoor test dataset.
- **Backdoor testing subset** D_{bs}^{test}. For the test dataset, we attack all non-target class graphs, which form the backdoor testing subset. This backdoor test subset is utilized to assess the metric–ASR.

5.2. Experiments settings

After introducing the dataset, to provide a more detailed description of our experiments, we introduce evaluation metrics, baseline methods, and parameter settings.

5.2.1. Evaluation metrics

We employ two evaluation metrics in this paper: Attack Success Rate (ASR) and Clean Accuracy Drop (CAD). ASR assesses the effectiveness of the attack methods, while CAD measures the stealth of these methods. We will provide detailed explanations of these two metrics below.

Attack Success Rate (ASR). Given a subset of backdoor tests $D_{bs}^{test} = \{G_1, G_2, \ldots, G_m\}$ where the class labels are assigned as 0. The ASR is defined as the proportion of graphs in D_{bs}^{test} that are predicted as target class labels by the backdoor classifier $\Phi_{Backdoor}$, i.e., ASR $= \frac{\sum_{i=1}^{m} \mathbb{I}(\Phi_{Backdoor}(f_{readout}(\mathrm{GConv}(G_i)))=l_{target})}{m}$ where $\mathbb{I}$ is the indicator function, and the result is 1 if $\Phi_{Backdoor}\left(f_{readout}(\mathrm{GConv}(G_i))\right)$ is equal to l_{target}.

Clean Accuracy Drop (CAD). CAD is the absolute difference between Clean Accuracy (CA) and Backdoor Accuracy (BA). Given $D_c^{test} = \{G_1, G_2, \ldots, G_m\}$ with labels $\{0, 1\}$, where CA is defined as the proportion of graphs in D_c^{test} predicted to be ground truth by the clean classifier $\Phi_{Clean}(\cdot)$, i.e., CA $= \frac{\sum_{i=1}^{m} \mathbb{I}(\Phi_{Clean}(f_{readout}(\mathrm{GConv}(G_i)))=l_{target})}{m}$; BA is defined as the proportion of graphs in D_c^{test} predicted to be ground truth class label by the backdoor classifier $\Phi_{Backdoor}(\cdot)$, i.e., BA $= \frac{\sum_{i=1}^{m} \mathbb{I}(\Phi_{Backdoor}(f_{readout}(\mathrm{GConv}(G_i)))=l_{target})}{m}$; So, CAD can be formalized as CAD $= |\mathrm{CA} - \mathrm{BA}|$.

5.2.2. Baselines

To demonstrate the effectiveness, we compare ACS with the following methods:

- **Backdoor Attacks to Graph Neural Networks** (Zhang et al., 2021): This is the first work based on subgraph backdoor attacks. It utilizes the ER model, SW model, and PA model to generate triggers and randomly inserts them into the graph.

- **Motif-Backdoor: Rethinking the Backdoor Attack on Graph Neural Networks via Motifs** (Zheng et al., 2023): This is a backdoor attack based on motifs. Motifs are statistically significant subgraphs that appear repeatedly in a graph, containing rich structural information. They use motifs with low frequency of occurrence in the graph as triggers to execute the backdoor attack.
- **Graph Backdoor** (Xi et al., 2021). Compared to previous approaches, Graph Backdoor stands out in several key ways: it is graph-oriented, defining triggers as specific subgraphs that include both topological structures and descriptive features, offering attackers a broad design space; it is input-tailored, dynamically adjusting triggers to individual graphs, thus enhancing attack effectiveness and evasiveness.
- **Clean-label Backdoor Attack on Graph Neural Networks** (Xu and Picek, 2022): This method shares similarities with **Backdoor Attacks to Graph Neural Networks**. But it injects randomly generated triggers only into graphs belonging to the same class as the attacker-chosen target class.
- **ACS-without-CL**: This is a variant of the ACS method. During the training phase, the contrastive learning technique is not applied following the utilization of the crucial substructure detection module on the attacker-chosen target class graphs. This divergence results in less similarity in the shapes of the crucial substructures among the target class graphs.

5.2.3. Parameter settings

Our attack is divided into two modules. The crucial substructure detection module: finding the crucial substructure of the target class graphs; The graph alignment module: modifying the crucial substructures of the non-target class graphs. The crucial substructure detection module: learning rate $lr_1 = 0.0001$, crucial substructure size $K = 4$. The graph alignment module: learning rate $lr_2 = 0.01$, attacker-chosen target class $l_{target} = 0$, poisoning rate $\rho = 0.01$. Epochs = 100 for both modules. GIN (Xu et al., 2018) is often targeted in backdoor attacks against graph classification tasks due to its effectiveness to capture details about diverse graph structures. Consequently, we have chosen GIN as our classifier. For pooling over nodes in a graph, we employ the 'sum' operation, and for pooling over neighboring nodes, we also utilize the 'sum' operation. The output dimension corresponds to the number of classes. Specifically, the classifier trained on the benign training dataset is referred to as the clean classifier $\Phi_{Clean}(\cdot)$. Conversely, the classifier trained on the backdoor training dataset is referred to as the backdoor classifier $\Phi_{Backdoor}(\cdot)$. The final result is the average value obtained after three runs of ACS. All experiments run on a computer with Intel Xeon E5-2640 CPU and 128 GB RAM, which installs Linux CentOS 7.1. For Graph Backdoor and motif-backdoor: The source codes for both methods are available as open-source. We ran their codes on machines with identical configurations, utilizing a poisoning rate of 0.01. Throughout the experiments, we maintained GIN as the classifier being targeted for the attack.

5.3. Performance comparison (RQ1)

We compare the performance of all methods on all four datasets.

Overall Comparison. Table 4 reports the performance comparison results. We have the following observations:

- *Backdoor Attacks to GNNs* achieves poor performance on MUTAG and Bitcoin. This indicates that simply randomly generating triggers and injecting them into non-target graphs is insufficient to force the classifier to associate the trigger with the attacker-chosen target class, thereby further limiting the performance. *Motif-backdoor* consistently outperforms *Backdoor Attacks to GNNs* on MUTAG and Bitcoin, demonstrating the importance of injecting triggers only into graphs belonging to the attacker-chosen target class graphs. *Graph Backdoor* outperforms Motif-backdoor on the MUTAG and Twitter datasets but slightly underperforms on the other two datasets. Moreover, *Graph Backdoor* achieves the best results on the MUTAG dataset, possibly due to its ability to adjust triggers based on the input.
- Compared to *Backdoor Attacks to GNNs*, *motif-backdoor* and *Graph Backdoor*, *Clean-label Backdoor Attack on GNNs* performs better on Twitter and Bitcoin, demonstrating the importance of injecting triggers only into graphs belonging to the attacker-chosen target class graphs. *ACS-without-CL* performs better on the Twitter and Bitcoin datasets, the performance of *ACS-without-CL* verifies that the crucial substructure detection module and the graph alignment module can improve ASR and reduce CAD. However, in Bicoin, *ACS-without-CL* underperforms *motif-backdoor* and *Graph Backdoor* w.r.t. ASR. The reason for this might be that *ACS-without-CL* does not utilize a contrastive learning technique, resulting in diverse shapes for the crucial substructures identified in the target class graphs and a lack of representativeness. *Motif-backdoor* selects low-frequency motifs as triggers, while *Clean-label Backdoor Attack on GNNs* directly injects generated triggers into target class graphs. However, both methods involve random triggers and injection locations, which can lead to slightly lower performance. To enhance their effectiveness, an adaptive approach to constructing triggers could be employed, ensuring that each target graph has a corresponding trigger. Additionally, choosing appropriate trigger injection locations in poisoned graphs, rather than injecting randomly, could improve performance.
- *ACS* achieves the best performance on most datasets, with an average ASR of 90.77%, while maintaining good stealth. By applying the crucial substructure detection module and the graph alignment module, *ACS* is capable of locating the crucial substructure as a trigger, and the injection process is completed by modifying the crucial substructure in the non-target class graphs. This highlights the significance of using crucial substructures as triggers and modifying them within non-target graphs as the injection positions. Moreover, compared with *ACS-without-CL*, *ACS* considers contrastive learning technique. This demonstrates that the contrastive learning technique can enhance the similarity in the shapes of crucial substructures among the target class graphs, thus enhancing their representativeness.

5.4. Study of ACS (RQ2)

Considering that the poisoning rate (ρ) and the size of the crucial substructure (K) are critical factors in the performance of *ACS*, we are examining their influence on performance using the MUTAG and Twitter dataset. We start by exploring the influence of ρ and then study how K affects the performance.

5.4.1. Effect of poisoning rate ρ

To investigate whether *ACS* are influenced by ρ, we vary the value of ρ. In particular, we search the ρ in the range of $\{1\%, 5\%, 10\%, 20\%, 50\%\}$. Fig. 9 summarizes the experimental results, we have the following observations:

- High ASR: As depicted by the black (MUTAG) and red (Twitter) line in Fig. 9, On both datasets, ASR consistently maintains a high ASR, exceeding 90%. This demonstrates the effectiveness of ACS.
- Correlation between ρ and ASR: As depicted in Fig. 9, on both datasets,we observe a positive correlation between ρ and ASR. As ρ increases, ASR also increases. The likely reason for this is that with the rise in poisoning rate, there is a more pronounced connection between the trigger and the target class. Consequently, the classifier finds it simpler to classify graphs containing the trigger into the target class.

Table 4
Overall performance of backdoor attacks. ('↑' indicates that a larger ASR value signifies a better attack effect, while '↓' indicates that a smaller CAD value signifies better attack stealthiness.)

Method	MUTAG	Twitter	Bitcoin	COLLAB
	↑ ASR/CAD ↓			
Backdoor attacks to GNNs	0.625/0.033	0.791/0.023	0.773/0.021	0.743/0.035
Motif-backdoor	0.818/0.053	0.361/0.033	0.895/0.034	0.559/0.117
Graph backdoor	**0.982**/0.052	0.591/0.091	0.878/0.072	0.539/0.102
Clean-label backdoor attack on GNNs	0.815/0.025	0.853/0.036	0.842/0.054	0.832/0.074
ACS-without-CL	0.872/0.046	0.881/0.025	0.861/0.046	0.857/0.030
ACS	0.906/0.034	**0.955**/0.016	**0.910**/0.020	**0.860**/0.027

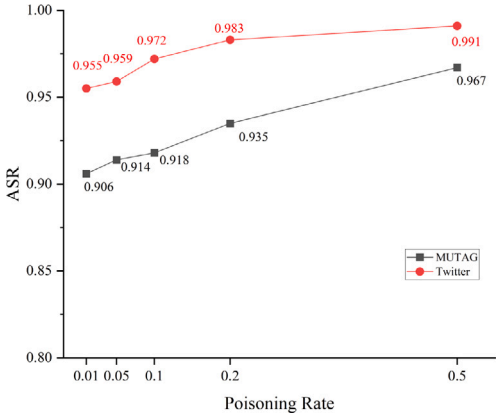


Fig. 9. Impact of poisoning rate.

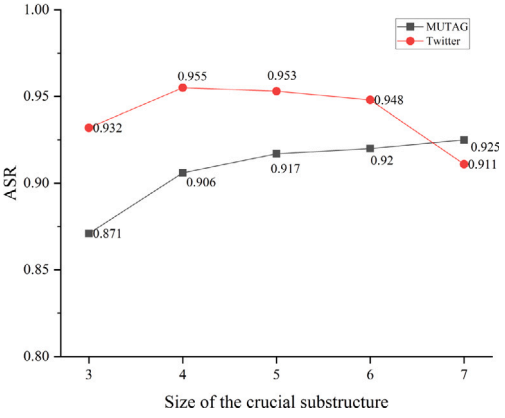


Fig. 10. The effect of the size of the crucial substructure on the ASR.

5.4.2. Effect of the size of the crucial substructure K

To investigate whether *ACS* are influenced by K, we vary the value of K. In particular, we search the K in the range of $\{3, 4, 5, 6, 7\}$ on both datasets. Fig. 10 summarizes the experimental results, we have the following observations: First, on Twitter dataset, as the number of edges in the crucial substructure increases, the ASR initially rises and then declines. As depicted in Fig. 10, when K exceeds 4, it may result in the inclusion of less crucial edges for the prediction outcome, leading to the crucial substructure becoming 'bloated'. Consequently, the classifier forms a weaker association between the crucial substructure and the target label. Second, on MUTAG dataset, as the size of the crucial substructure increases, so does the ASR. This is potentially due to the smaller scale of graphs in the MUTAG dataset. With the enlargement of the crucial substructure, the association between the trigger and the target class becomes more pronounced, leading to an increase in the ASR.

5.5. Effect of the crucial substructure detection module (RQ3)

We have contended that the identified crucial substructures frequently determine the class of the entire graph, thus, in this section, we validate the efficacy of the crucial substructure detection module and ACS attacks through two case studies conducted on the MUTAG dataset. The first case study evaluates the effectiveness of the crucial substructure detection module, while the second case study evaluates the effectiveness of ACS attacks.

Validation of the crucial substructure detection module. In the context of mutagenic prediction, it is well-established that the Nitrobenzene structure, characterized by its nitro elements and the nitro group NO_2 bonded to a carbon ring, is a primary determinant of mutagenicity (Li et al., 2023b). As illustrated in Fig. 11, our method effectively pinpoints the nitro group NO_2 attached to a carbon ring, which signifies the Nitrobenzene structure when $K = 9$. The numerical values assigned to these edges denote their relative importance, with lower numbers indicating greater significance. Consequently, the crucial substructure detection module of ACS precisely identifies the critical substructures within the graph.

Validation of ACS attack effectiveness. In this subsection, we endeavor to comprehend the entire attack process. To this end, we randomly select 27 graphs from the MUTAG dataset, comprising 13 graphs in class 1 (represented by blue circles) and 13 in class 0 (represented by red crosses). We then choose one graph in class 1 with ID 47 to perform backdoor attack and observe the changes in its embedding before and after the attack (depicted by green squares and yellow triangles, respectively). Fig. 12 presents the visualization results of the embeddings for these graphs. Two notable observations arise from this visualization: (1) Nodes from different classes exhibit distinct boundaries in the embedding space. (2) Pre-attack, the embedding of the graph with ID 47 aligns closely with the position of class 1 nodes; post-attack, its embedding moves closer to the position of class 0 nodes.

These visualizations provide compelling evidence of our ACS's ability to identify crucial substructures, underscoring its practical effectiveness and relevance in real-world applications. Nevertheless, our conclusions are based on statistical correlations, identifying essential substructures rather than true causal factors. Future research could benefit from adopting advanced counterfactual analysis techniques to discern the genuine causal factors.

6. Conclusion

In this work, we introduce a novel attack method called ACS, which incorporates crucial substructures into backdoor attacks with remarkable ASR and CAD. Specifically, we present a crucial substructure detection module to identify crucial substructures associated with the desired prediction. We then modify the crucial substructures in target class graphs via contrastive learning to enhancing the representativeness. Besides, a graph alignment module is designed to manipulate the crucial substructures within the non-target class graphs. Experimental results on four datasets clearly demonstrate the superiority of ACS over baseline methods, particularly in terms of ASR and CAD.

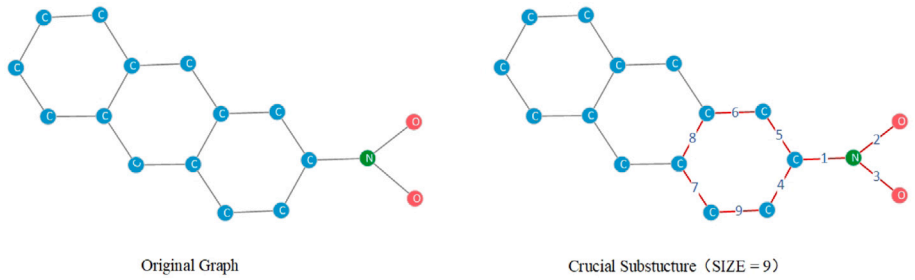


Fig. 11. Visualization of the result of ACS in MUTAG data, with crucial substructure of size 9. (Graph ID: 47).

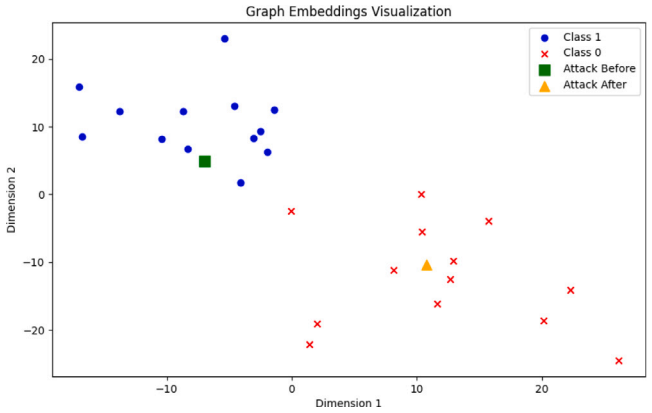


Fig. 12. Visualization of the result of ACS in MUTAG data (The orange and cyan dots in the figure represent the graph with ID 47).

It is crucial to recognize that our method for detecting crucial structures establishes correlations rather than establishing causal relationships. In other words, the crucial substructures identified by ACS suggest associations rather than definitive causal factors. Consequently, our future research will focus on utilizing advanced counterfactual inference techniques to pinpoint genuine causal factors. Moreover, an interesting research avenue is the development of defenses against backdoor attacks when dealing with adaptively extracted crucial substructures.

CRediT authorship contribution statement

Haibin Tong: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Huifang Ma:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition, Formal analysis. **Hui Shen:** Investigation, Formal analysis. **Zhixin Li:** Supervision, Funding acquisition. **Liang Chang:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: We would like to submit the manuscript entitled 'Crucial rather than Random: Attacking Crucial Substructure for Backdoor Attacks on Graph Neural Networks', which we wish to be considered for publication in Engineering Applications of Artificial Intelligence. No conflict of interest exits in the submission of this manuscript, and manuscript is approved by all authors for publication.

Data availability

All datasets are publicly available.

Acknowledgments

This study is supported by Gansu Province Key Foundation Project (No. 24JRRA123), the Industrial Support Project of Gansu Colleges, China (No. 2022CYZC11), National Natural Science Foundation of China (61762078, 62276073 and U22A2099), and the Graduate Research Support Project of Northwest Normal University (2023XYKYZZ016LX011)

References

Barabási, Albert-László, Albert, Réka, 1999. Emergence of scaling in random networks. Science 286 (5439), 509–512.

Chen, Xinyun, Liu, Chang, Li, Bo, Lu, Kimberly, Song, Dawn, 2017. Targeted backdoor attacks on deep learning systems using data poisoning. arXiv preprint arXiv:1712.05526.

Dai, Enyan, Lin, Minhua, Zhang, Xiang, Wang, Suhang, 2023. Unnoticeable backdoor attacks on graph neural networks. In: Proceedings of the ACM Web Conference 2023. pp. 2263–2273.

Debnath, Asim Kumar, Lopez de Compadre, Rosa L, Debnath, Gargi, Shusterman, Alan J, Hansch, Corwin, 1991. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. correlation with molecular orbital energies and hydrophobicity. J. Med. Chem. 34 (2), 786–797.

Ding, Hua, Chen, Lixing, Li, Shenghong, Bai, Yang, Zhou, Pan, Qu, Zhe, 2024. Divide, conquer, and coalesce: Meta parallel graph neural network for IoT intrusion detection at scale. In: Proceedings of the ACM on Web Conference 2024. pp. 1656–1667.

Doan, Khoa, Lao, Yingjie, Zhao, Weijie, Li, Ping, 2021. Lira: Learnable, imperceptible and robust backdoor attacks. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 11966–11976.

Gao, Zihao, Ma, Huifang, Zhang, Xiaohui, Wang, Yike, Wu, Zheyu, 2023a. Similarity measures-based graph co-contrastive learning for drug–disease association prediction. Bioinformatics 39 (6), btad357.

Gao, Chen, Zheng, Yu, Li, Nian, Li, Yinfeng, Qin, Yingrong, Piao, Jinghua, Quan, Yuhan, Chang, Jianxin, Jin, Depeng, He, Xiangnan, et al., 2023b. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. ACM Trans. Recomm. Syst. 1 (1), 1–51.

Gu, Tianyu, Dolan-Gavitt, Brendan, Garg, Siddharth, 2017. Badnets: Identifying vulnerabilities in the machine learning model supply chain. arXiv preprint arXiv:1708.06733.

Huang, Taoan, Ferber, Aaron M, Tian, Yuandong, Dilkina, Bistra, Steiner, Benoit, 2023. Searching large neighborhoods for integer linear programs with contrastive learning. In: International Conference on Machine Learning. PMLR, pp. 13869–13890.

Janson, Svante, Luczak, Tomasz, Rucinski, Andrzej, 2011. Random Graphs. John Wiley & Sons.

Li, Yuezun, Li, Yiming, Wu, Baoyuan, Li, Longkang, He, Ran, Lyu, Siwei, 2021. Invisible backdoor attack with sample-specific triggers. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 16463–16472.

Li, Qingfeng, Ma, Huifang, Zhang, Ruoyi, Jin, Wangyu, Li, Zhixin, 2023a. Intra- and inter-behavior contrastive learning for multi-behavior recommendation. In: International Conference on Database Systems for Advanced Applications. Springer, pp. 147–162.

Li, Jianxin, Sun, Qingyun, Peng, Hao, Yang, Beining, Wu, Jia, Phillp, S Yu, 2023b. Adaptive subgraph neural network with reinforced critical structure mining. IEEE Trans. Pattern Anal. Mach. Intell..

Li, Shuya, Wan, Fangping, Shu, Hantao, Jiang, Tao, Zhao, Dan, Zeng, Jianyang, 2020. MONN: a multi-objective neural network for predicting compound-protein interactions and affinities. Cell Syst. 10 (4), 308–322.

Li, Wenshuo, Yu, Jincheng, Ning, Xuefei, Wang, Pengjun, Wei, Qi, Wang, Yu, Yang, Huazhong, 2018. Hu-fu: Hardware and software collaborative attack framework against neural networks. In: 2018 IEEE Computer Society Annual Symposium on VLSI. ISVLSI, IEEE, pp. 482–487.

Lu, Yen-Wen, Tsai, Yu-Che, Li, Cheng-Te, 2024. Burstiness-aware bipartite graph neural networks for fraudulent user detection on rating platforms. In: Companion Proceedings of the ACM on Web Conference 2024. pp. 834–837.

Manu, Daniel, Yao, Jingjing, Liu, Wuji, Sun, Xiang, 2024. GraphGANFed: A federated generative framework for graph-structured molecules towards efficient drug discovery. IEEE/ACM Trans. Comput. Biol. Bioinform..

Nachmani, Eliya, Wolf, Lior, 2020. Molecule property prediction and classification with graph hypernetworks. arXiv preprint arXiv:2002.00240.

Nguyen, Anh, Tran, Anh, 2021. Wanet–imperceptible warping-based backdoor attack. arXiv preprint arXiv:2102.10369.

Piech, Chris, Bassen, Jonathan, Huang, Jonathan, Ganguli, Surya, Sahami, Mehran, Guibas, Leonidas J, Sohl-Dickstein, Jascha, 2015. Deep knowledge tracing. Adv. Neural Inf. Process. Syst. 28.

Saha, Aniruddha, Tejankar, Ajinkya, Koohpayegani, Soroush Abbasi, Pirsiavash, Hamed, 2022. Backdoor attacks on self-supervised learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 13337–13346.

Schafer, J Ben, Frankowski, Dan, Herlocker, Jon, Sen, Shilad, 2007. Collaborative filtering recommender systems. In: The Adaptive Web: Methods and Strategies of Web Personalization. Springer, pp. 291–324.

Sun, Fan-Yun, Hoffmann, Jordan, Verma, Vikas, Tang, Jian, 2019. Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. arXiv preprint arXiv:1908.01000.

Sun, Li, Zhang, Zhongbao, Wang, Feiyang, Ji, Pengxin, Wen, Jian, Su, Sen, Philip, S Yu, 2022. Aligning dynamic social networks: An optimization over dynamic graph autoencoder. IEEE Trans. Knowl. Data Eng. 35 (6), 5597–5611.

Tran, Brandon, Li, Jerry, Madry, Aleksander, 2018. Spectral signatures in backdoor attacks. Adv. Neural Inf. Process. Syst. 31.

Veličković, Petar, Fedus, William, Hamilton, William L, Liò, Pietro, Bengio, Yoshua, Hjelm, R Devon, 2018. Deep graph infomax. arXiv preprint arXiv:1809.10341.

Wang, Xiao, Liu, Nian, Han, Hui, Shi, Chuan, 2021. Self-supervised heterogeneous graph neural network with co-contrastive learning. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. pp. 1726–1736.

Wang, Wentao, Ma, Huifang, Zhao, Yan, Yang, Fanyi, Chang, Liang, 2022. SEEP: Semantic-enhanced question embeddings pre-training for improving knowledge tracing. Inform. Sci. 614, 153–169.

Wang, Binghui, Zhang, Le, Gong, Neil Zhenqiang, 2017. SybilSCAR: Sybil detection in online social networks via local rule based propagation. In: IEEE INFOCOM 2017-IEEE Conference on Computer Communications. IEEE, pp. 1–9.

Watts, Duncan J., Strogatz, Steven H., 1998. Collective dynamics of 'small-world'networks. Nature 393 (6684), 440–442.

Weber, Mark, Domeniconi, Giacomo, Chen, Jie, Weidele, Daniel Karl I, Bellei, Claudio, Robinson, Tom, Leiserson, Charles E, 2019. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. arXiv preprint arXiv:1908.02591.

Xi, Zhaohan, Pang, Ren, Ji, Shouling, Wang, Ting, 2021. Graph backdoor. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 1523–1540.

Xiao, Zhiwen, Tong, Huagang, Qu, Rong, Xing, Huanlai, Luo, Shouxi, Zhu, Zonghai, Song, Fuhong, Feng, Li, 2023. CapMatch: Semi-supervised contrastive transformer capsule with feature-based knowledge distillation for human activity recognition. IEEE Trans. Neural Netw. Learn. Syst..

Xiao, Zhiwen, Xing, Huanlai, Qu, Rong, Feng, Li, Luo, Shouxi, Dai, Penglin, Zhao, Bowen, Dai, Yuanshun, 2024a. Densely knowledge-aware network for multivariate time series classification. IEEE Trans. Syst. Man Cybern.: Syst..

Xiao, Zhiwen, Xu, Xin, Xing, Huanlai, Zhao, Bowen, Wang, Xinhan, Song, Fuhong, Qu, Rong, Feng, Li, 2024b. DTCM: Deep transformer capsule mutual distillation for multivariate time series classification. IEEE Trans. Cogn. Dev. Syst..

Xing, Xiaogang, Xu, Ming, Bai, Yujing, Yang, Dongdong, 2023. A clean-label graph backdoor attack method in node classification task. arXiv preprint arXiv:2401.00163.

Xu, Keyulu, Hu, Weihua, Leskovec, Jure, Jegelka, Stefanie, 2018. How powerful are graph neural networks? arXiv preprint arXiv:1810.00826.

Xu, Jing, Picek, Stjepan, 2022. Poster: Clean-label backdoor attack on graph neural networks. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 3491–3493.

Xu, Jing, Picek, Stjepan, 2023. Poster: Multi-target & multi-trigger backdoor attacks on graph neural networks. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 3570–3572.

Yanardag, Pinar, Vishwanathan, SVN, 2015. Deep graph kernels. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1365–1374.

Yang, Xiao, Li, Gaolei, Zhang, Chaofeng, Han, Meng, Yang, Wu, 2023. Per-CBA: Persistent clean-label backdoor attacks on semi-supervised graph node classification.

Ying, Zhitao, Bourgeois, Dylan, You, Jiaxuan, Zitnik, Marinka, Leskovec, Jure, 2019. Gnnexplainer: Generating explanations for graph neural networks. Adv. Neural Inf. Process. Syst. 32.

Zhang, Hangfan, Chen, Jinghui, Lin, Lu, Jia, Jinyuan, Wu, Dinghao, 2023. Graph contrastive backdoor attacks. In: International Conference on Machine Learning. PMLR, pp. 40888–40910.

Zhang, Zaixi, Jia, Jinyuan, Wang, Binghui, Gong, Neil Zhenqiang, 2021. Backdoor attacks to graph neural networks. In: Proceedings of the 26th ACM Symposium on Access Control Models and Technologies. pp. 15–26.

Zhao, Yan, Ma, Huifang, Wang, Wentao, Gao, Weiwei, Yang, Fanyi, He, Xiangchun, 2023. Exploiting multiple question factors for knowledge tracing. Expert Syst. Appl. 223, 119786.

Zheng, Haibin, Xiong, Haiyang, Chen, Jinyin, Ma, Haonan, Huang, Guohan, 2023. Motif-backdoor: Rethinking the backdoor attack on graph neural networks via motifs. IEEE Trans. Comput. Soc. Syst..